Event-Based Architecture Types: Complete Guide with Examples

What is Event-Based Architecture?

Event-based architecture is a design pattern where components communicate by producing and consuming events. When something happens (an event), it triggers actions in other parts of the system.

Understanding the Classification Matrix

The matrix classifies event-based systems based on two dimensions:

Referential Coupling

- **Coupled**: Components know about each other directly
- **Decoupled**: Components don't know about each other directly

Temporal Coupling

- **Coupled**: Components must be active at the same time
- **Decoupled**: Components can be active at different times

---

1. Direct Communication

Referentially Coupled + Temporally Coupled

Characteristics:

- Components know each other directly
- Both sender and receiver must be active simultaneously
- Direct method calls or function invocations

Real-World Example: Restaurant Kitchen

```
Cook → "Order ready!" → Waiter
```

Programming Example:

```
java

class OrderService {
    private NotificationService notificationService;

    public void processOrder(Order order) {
        // Process the order
        order.setStatus("COMPLETED");

        // Direct call - both services must be running
        notificationService.sendNotification(order.getCustomerId(),
                                             "Your order is ready!");
    }
}
```

When to Use:

- Simple, small applications
- When you need immediate response
- Tight integration between components

Limitations:

- If one component fails, the whole chain breaks
- Hard to scale individual components
- Components are tightly bound together

---

2. Mailbox System

Referentially Coupled + Temporally Decoupled

Characteristics:

- Components know specific targets
- Messages can be delivered even if receiver is offline
- Messages are stored until receiver is ready

Real-World Example: Email System

```
You send email to john@company.com → Email sits in John's mailbox → John reads
it later
```

Programming Example:

java

```java
class OrderService {
    private MailboxService mailboxService;

    public void processOrder(Order order) {
        order.setStatus("COMPLETED");

        // Send to specific user's mailbox
        Message msg = new Message(order.getCustomerId(),
                                  "Order completed",
                                  order.getDetails());
        mailboxService.sendToMailbox("user_" + order.getCustomerId(), msg);
    }
}

class NotificationService {
    public void checkMailbox() {
        // Can check mailbox anytime, even if OrderService is offline
        List<Message> messages = mailboxService.getMessages("notification_service'
        for (Message msg : messages) {
            processNotification(msg);
        }
    }
}
```

When to Use:

- When receiver might be temporarily unavailable
- Asynchronous processing needed
- Point-to-point communication

Examples:

- Message queues (RabbitMQ, ActiveMQ)
- Email systems
- SMS services

---

3. Event-Based (Publish-Subscribe)

Referentially Decoupled + Temporally Coupled

Characteristics:

- Publishers don't know who will receive events
- Subscribers don't know who sent events
- All parties must be active simultaneously
- Events are broadcast to all interested parties

Real-World Example: Radio Broadcasting

```
Radio Station broadcasts → Anyone tuned to that frequency receives the signal
```

Programming Example:

java

```java
class OrderService {
    private EventBus eventBus;

    public void processOrder(Order order) {
        order.setStatus("COMPLETED");

        // Publish event - doesn't know who's listening
        OrderCompletedEvent event = new OrderCompletedEvent(order);
        eventBus.publish("order.completed", event);
    }
}

class NotificationService {
    public NotificationService(EventBus eventBus) {
        // Subscribe to events
        eventBus.subscribe("order.completed", this::handleOrderCompleted);
    }

    private void handleOrderCompleted(OrderCompletedEvent event) {
        sendNotification(event.getOrder());
    }
}

class AnalyticsService {
    public AnalyticsService(EventBus eventBus) {
        // Same event, different handler
        eventBus.subscribe("order.completed", this::trackOrderMetrics);
    }

    private void trackOrderMetrics(OrderCompletedEvent event) {
```

```
            updateSalesMetrics(event.getOrder());
        }
    }
```

When to Use:

- Multiple components need to react to same event
- Real-time notifications
- Loose coupling between components

Examples:

- DOM events in web browsers
- GUI event systems
- Real-time chat applications
- Live sports score updates

---

4. Shared Data Space (Tuple Space)

Referentially Decoupled + Temporally Decoupled

Characteristics:

- Components don't know each other
- Components can operate at different times
- Data/events are stored in shared space
- Components read/write to shared space when convenient

Real-World Example: Library Bulletin Board

```
Someone posts a "Roommate Wanted" note → Note stays on board →
Someone else reads it days later and responds
```

Programming Example:

java

```java
class OrderService {
    private SharedDataSpace dataSpace;

    public void processOrder(Order order) {
        order.setStatus("COMPLETED");

        // Put data in shared space - don't know who will read it
        OrderTuple tuple = new OrderTuple("order_completed",
                                          order.getId(),
                                          order.getCustomerId(),
                                          System.currentTimeMillis());
        dataSpace.write(tuple);
    }
}

class NotificationService {
    private SharedDataSpace dataSpace;

    public void processNotifications() {
        // Read when convenient - might be hours later
        OrderTuple template = new OrderTuple("order_completed", null, null, null)

        while (true) {
            OrderTuple tuple = dataSpace.take(template); // Blocks until available
            if (tuple != null) {
                sendNotification(tuple.getCustomerId(),
                                 "Order " + tuple.getOrderId() + " completed");
            }
        }
    }
```

```
}
```

When to Use:

- Batch processing systems
- Data pipeline architectures
- Systems with intermittent connectivity
- Workflow management

Examples:

- Apache Kafka (with persistent topics)
- Database-based job queues
- File-based data exchange
- Blockchain systems
- Linda tuple spaces

Comparison Table

Type	Referential	Temporal	Example	Best For
Direct	Coupled	Coupled	Method calls	Simple, immediate responses
Mailbox	Coupled	Decoupled	Email, Message queues	Point-to-point async
Event-based	Decoupled	Coupled	Pub/Sub, Radio	Real-time notifications
Shared Data Space	Decoupled	Decoupled	Tuple spaces, Kafka	Batch processing, workflows

Real-World System Example: E-commerce Platform

Order Processing Flow:

1. **Direct**: Payment validation (immediate response needed)

   ```
   OrderService → PaymentService.validatePayment() → Immediate success/failure
   ```

2. **Mailbox**: Inventory updates (specific service, can be delayed)

   ```
   OrderService → InventoryService mailbox → Process when inventory service is ready
   ```

3. **Event-based**: Multiple immediate reactions

   ```
   OrderService publishes "OrderPlaced" →
   - EmailService sends confirmation
   - AnalyticsService updates metrics
   - RecommendationService triggers suggestions
   ```

4. **Shared Data Space**: Data warehouse updates (batch processing)

   ```
   OrderService writes order data → Data warehouse reads overnight → Reports generated
   ```

Choosing the Right Architecture

Use **Direct** when:

- Simple request-response needed
- Tight coupling is acceptable
- Immediate feedback required

Use **Mailbox** when:

- Specific point-to-point communication
- Receiver might be temporarily unavailable
- Message persistence needed

Use **Event-based** when:

- Multiple components react to same event
- Real-time notifications required
- Loose coupling desired

Use **Shared Data Space** when:

- Batch processing workflows
- Components operate at different schedules
- Maximum decoupling needed
- Data persistence across time required